

Práctica 3 SIW - Documentación

Pelayo Sierra Lobo UO294217

Simhash

El script implementado se puede encontrar en el archivo `simhash-practice.py`. Utiliza las librerías *Simhash* y *SimhashIndex* para detectar documentos cuasi-duplicados dentro de colecciones.

Cada función contiene comentarios sobre el funcionamiento general y propósito del código. Para ejecutarlo, solo hay que modificar la "Zona de experimentación" al final del script.

```
# ----- Zona de experimentación -----
- #
train_number = 1000
f = 128
quora = False
if quora:
    sentences = organize_sentences_quora()
else:
    sentences = organize_sentences_mccornick(train_number)
words = create_obj_words(sentences)
tolerances = [0, 3, 5, 10]

for i in range(len(tolerances)):
    if os.path.exists(f'simhash/results/results_{tolerances[i]}.txt'):
        os.remove(f'simhash/results/results_{tolerances[i]}.txt')
    index_words = SimhashIndex(words, k=tolerances[i], f=f)
    written_pairs = compare(sentences, tolerances[i])
    if quora:
        truths_file = "simhash/quora/duplicates.txt"
    else:
        truths_file = f"simhash/truth/articles_{train_number}.truth"
    VP, FP, FN = compute_values(written_pairs, parse_truths(truths_file))
    precision, recall = compute_precision_recall(VP, FP, FN)
    F1 = compute_F1(precision, recall)

    print(f"Tolerance      = {tolerances[i]}")
    print(f"VP              = {VP}")
    print(f"FP              = {FP}")
    print(f"FN              = {FN}")
    print(f"Precisión       = {precision}")
    print(f"Exhaustividad   = {recall}")
```

```
print(f"F1 = {F1}")
print("-----")
```

Se puede experimentar con ambas colecciones de la especificación. En caso de querer utilizar la colección de McCornick, el atributo `quora` debe ser `False`. De lo contrario, se utilizará la colección de Quora. El `train_number` se utiliza para especificar el archivo concreto dentro de la colección de McCornick, debe de ser un numero de los siguientes: [100, 1000, 10000, 2500].

Note

No hay manejo de errores a la hora de introducir los valores de los atributos, ya que no consideré que fuera uno de los objetivos de la práctica. Lo mismo ocurre con el uso del atributo `quora`, no es un alcance extensible, pero me quedé sin "presupuesto" de horas para hacer algo más sofisticado.

El propio script se encarga de extraer la información adecuada tanto para la detección de duplicados como la comparativa mediante precisión, exhaustividad y F1 (mediante distintos archivos de "verdades"). En ambas colecciones, debemos encontrar el valor adecuado de `f` para evitar tener el error `Big Bucket` a la hora de hacer el hashing, siempre intentando que este valor sea el mínimo posible. Esto será mencionado más adelante, pero `f` siempre debe de ser `128` salvo para el caso de `articles_100`, que sirve con `f=64`.

Experimento

A continuación se adjuntan los resultados obtenidos tras experimentar con las distintas colecciones para valores de tolerancia `[0, 3, 5, 10]` y distintos valores de `f`, así como una reflexión acerca de estos.

Es importante recalcar que, durante la elaboración de este proyecto, detecté una limitación importante en el algoritmo de simhash: no es capaz de procesar valores de `f` mayores a 128 bits. Investigando un poco, he descubierto que el problema está en `SimhashIndex`. Con `f = 64` o `128`, se producen *digests* de 8 o 16 bytes. A valores mayores de esto, hay tamaños que generan longitudes no divisibles por `f`, de ahí que surjan errores como el siguiente:

```
ValueError: cannot reshape array of size 11904 into shape (256)
```

$11904 / 256 = 46.5$

Solucionar esto implicaría hacer una investigación en profundidad del código. Por conveniencia, limitaré los valores de `k` en los articles que produzcan `Big Bucket` con `f = 128`.

McCornick - articles_100

```
Tolerance      = 0
VP              = 2
FP              = 0
FN              = 3
Precisión      = 1.0
Exhaustividad  = 0.4
F1 = 0.5714285714285715
```

```
-----
Tolerance      = 3
VP              = 5
FP              = 0
FN              = 0
Precisión      = 1.0
Exhaustividad  = 1.0
F1 = 1.0
```

```
-----
Tolerance      = 5
VP              = 5
FP              = 8
FN              = 0
Precisión      = 0.38461538461538464
Exhaustividad  = 1.0
F1 = 0.5555555555555556
```

```
-----
Tolerance      = 10
VP              = 5
FP              = 519
FN              = 0
Precisión      = 0.009541984732824428
Exhaustividad  = 1.0
F1 = 0.01890359168241966
```

En este ejemplo se puede ver cómo el F1 ideal (1.0) es alcanzado con tolerancia 3. Esto implica que encuentra absolutamente a todos los duplicados establecidos en el archivo de verdades, sin falsos positivos ni falsos negativos. A valores mayores, se produce una relajación excesiva en la distancia de hamming que permite la aparición de un número muy alto de falsos duplicados.

McCornick - articles_1000

Para esta colección, debemos de aumentar el valor de `f` a 128 , de lo contrario se produce el error *Big Bucket*.

```
Tolerance      = 0
VP             = 3
FP             = 0
FN             = 7
Precisión      = 1.0
Exhaustividad = 0.3
F1 = 0.4615384615384615
```

```
-----
Tolerance      = 3
VP             = 10
FP             = 0
FN             = 0
Precisión      = 1.0
Exhaustividad = 1.0
F1 = 1.0
```

```
-----
Tolerance      = 5
VP             = 10
FP             = 0
FN             = 0
Precisión      = 1.0
Exhaustividad = 1.0
F1 = 1.0
```

```
-----
Tolerance      = 10
VP             = 10
FP             = 3
FN             = 0
Precisión      = 0.7692307692307693
Exhaustividad = 1.0
F1 = 0.8695652173913044
-----
```

En este caso, obtenemos $F1 = 1.0$ tanto para las tolerancias 3 como 5. Esto implica que, para esta colección, el cambio de bits de tolerancia no parece tener un efecto tan significativo como en el escenario anterior. De hecho, hasta para una tolerancia de 10 bits podemos observar un valor de F1 considerablemente grande, con una precisión de aproximadamente un 77%.

McCornick - articles_10000

Eliminamos los valores para k superiores a 3, de modo que `tolerances = [0, 3]` para evitar errores Big Bucket.

```
Tolerance      = 0
VP             = 56
FP             = 0
```

```
FN          = 24
Precisión   = 1.0
Exhaustividad = 0.7
F1 = 0.8235294117647058
```

```
Tolerance   = 3
VP          = 76
FP          = 0
FN          = 4
Precisión   = 1.0
Exhaustividad = 0.95
F1 = 0.9743589743589743
```

Como el valor 5 producía este error y $k=3$ se quedaba muy cerca del $F1=1.0$, decidí probar a añadir $k=4$ para determinar si esta tolerancia daría resultados mejores:

```
Tolerance   = 4
VP          = 80
FP          = 0
FN          = 0
Precisión   = 1.0
Exhaustividad = 1.0
F1 = 1.0
```

Se puede observar que, para esta colección, conseguimos que los resultados sean exactamente los mismos que en el archivo `.truth` con valores de $k=4$.

McCornick - articles_2500

Similar al caso anterior, en este caso `tolerances = [0, 3, 5]`.

```
Tolerance   = 0
VP          = 5
FP          = 0
FN          = 15
Precisión   = 1.0
Exhaustividad = 0.25
F1 = 0.4
```

```
Tolerance   = 3
VP          = 18
FP          = 0
FN          = 2
Precisión   = 1.0
Exhaustividad = 0.9
```

```
F1 = 0.9473684210526316
```

```
-----  
Tolerance      = 5  
VP             = 20  
FP             = 0  
FN             = 0  
Precisión      = 1.0  
Exhaustividad  = 1.0  
F1 = 1.0  
-----
```

El valor máximo de F1 se alcanza con un valor de tolerancia 5. De nuevo, quise experimentar con el valor máximo de k permitido antes de provocar Big Bucket, en este caso k=6 produce los mismos resultados que 5 y es el máximo:

```
-----  
Tolerance      = 6  
VP             = 20  
FP             = 0  
FN             = 0  
Precisión      = 1.0  
Exhaustividad  = 1.0  
F1 = 1.0  
-----
```

Quora

El último experimento se hizo sobre la colección de Quora. Esta colección sufre la misma limitación mencionada antes con respecto al valor de f y las tolerancias, por lo que se mantiene un valor de $f=128$. Tras experimentar, el valor máximo de k para esta colección sería 7, por lo que `tolerances = [0, 3, 5, 7]`.

```
Tolerance      = 0  
VP             = 322  
FP             = 16  
FN             = 64524  
Precisión      = 0.9526627218934911  
Exhaustividad  = 0.00496561083181692  
F1 = 0.009879725085910653  
-----  
Tolerance      = 3  
VP             = 327  
FP             = 17  
FN             = 64519  
Precisión      = 0.9505813953488372  
Exhaustividad  = 0.005042716590074946  
F1 = 0.010032213529682466
```

```
-----  
Tolerance      = 5  
VP             = 354  
FP            = 19  
FN            = 64492  
Precisión     = 0.9490616621983914  
Exhaustividad = 0.005459087684668291  
F1 = 0.010855732225271777  
-----
```

```
-----  
Tolerance      = 7  
VP             = 443  
FP            = 28  
FN            = 64403  
Precisión     = 0.940552016985138  
Exhaustividad = 0.0068315701816611665  
F1 = 0.013564615643706845  
-----
```

Los valores de F1 son notablemente bajos, siendo muy considerable la cantidad de **falsos negativos**. Sí que se puede observar que, por debajo de $F1=1.0$, a mayor valor de distancia de hamming (k), mayor valor de F1.

Reflexión sobre los resultados

Es evidente que la diferencia de los resultados de ambas colecciones es inmensa. El motivo de esto se basa en la propia naturaleza de los contenidos de las colecciones. Las de McCornick contienen documentos prácticamente iguales. Por ejemplo, para `articles_100` encontramos la siguiente cuasi-duplicación: `t980 t2023`. Utilizando la herramienta [Text Compare](#), observamos que sólo se diferencian en que una utiliza la palabra "millions" y la otra no. Esto supone una coincidencia casi del 100% entre ambos textos.

Sin embargo, las preguntas de Quora difieren mucho a nivel léxico. La primera coincidencia que nos sugiere el archivo `duplicates.txt` es entre las preguntas 37 y 38:

```
37 Why are so many Quora users posting questions that are readily answered  
on Google?  
38 Why do people ask Quora questions which can be answered easily by  
Google?
```

El significado es el mismo, como humanos es fácil comprender que son prácticamente la misma pregunta. Sin embargo, Simhash no utiliza significados, se basa exclusivamente en aproximaciones léxicas. Al transformar ambas preguntas a una secuencia de bits, probablemente difieran tanto que requieren de distancias de hamming exorbitadas, algo contraproducente a la hora de buscar documentos duplicados. Esto se hace evidente al ver las coincidencias que encuentra nuestro algoritmo de búsqueda (en el archivo `results`). Por ejemplo, para $k=7$ encontramos `144` y `462802`, que es un claro duplicado:

144 What is narcissistic personality disorder?

462802 What is Narcissistic personality disorder?

O 238 y 448184 , que se acerca notablemente a nivel léxico:

238 What are some mind-blowing technologies that exist that most people don't know about?

448184 What are some mind-blowing car help tools that exist that most people don't know about?

Este último caso es parecido a nivel léxico, pero es a la vez un falso positivo, ya que ambas preguntas tratan de temas suficientemente diferentes como para no ser consideradas duplicados a nivel semántico.

Esto demuestra que la detección de duplicados no ha de ser analizada exclusivamente desde un punto de vista léxico, sino semántico, ya que palabras muy distintas y oraciones redactadas de maneras muy diferentes pueden conllevar el mismo significado y por lo tanto ser un "duplicado" a ojos del usuario.